

知识表示与推理

命题逻辑与谓词逻辑

命题逻辑

定义 命题：确定为真或为假的陈述句

- 原子命题：不包含其他命题作为其组成部分的命题，又称简单命题。
- 只有具有确定真值的陈述句才是命题，无法判断真或假的描述性句子不能作为命题

定义 复合命题：包含其他命题作为其组成部分的命题

- 命题联结词： $\wedge \vee \neg \rightarrow \leftrightarrow$

定义 逻辑等价：在所有情况下都具有相同的真假结果，一般用 $\equiv$ 来表示

谓词逻辑

定义 谓词逻辑：刻画主体(个体和群体)之间逻辑关系的方法

- 一元谓词
 - 包含一个参数
 - 用于刻画个体是否包含特定的属性
 - $P(x)$ ： x 是质数，表示某个数是否是质数
- 多元谓词
 - 包含多个参数
 - 用于表示个体之间的多元关系
 - $Father(x, y)$ ： x 是 y 的父亲
- 从自然语言到谓词描述
 - Richard是国王 $King(Richard)$
 - Lucy和Lily是姐妹 $Sister(Lucy, Lily)$
 - 北京是中国的首都 $Capital(北京, 中国)$

定义 全称量词和存在量词

- 全称量词：全称量词表示一切的、所有的、凡是、每一个等，用符号 $\forall$ 表示
- 存在量词：存在量词表示存在、有一个、某些等，用符号 $\exists$ 表示
- 从自然语言到谓词描述
 - 所有的国王都是人 $(\forall x)(King(x) \rightarrow Person(x))$
 - 所有的国王都头戴皇冠 $(\forall x)(King(x) \rightarrow HeadOn(Crown, x))$

归结原理及应用

一阶逻辑化为子句

定义 子句：组成合取范式的析取式

定义 空子句：不包含任何文字的子句，表示为NIL

将一阶逻辑公式化为子句集

- 以 $(\forall x)\{[\neg P(x) \vee \neg Q(x)] \rightarrow (\exists y)[S(x, y) \wedge Q(x)]\} \wedge (\forall x)[P(x) \vee B(x)]$ 为例

1. 消去蕴含和等价

- $P \rightarrow Q = \neg P \vee Q$
- $P \rightarrow Q = (P \wedge Q) \vee (\neg P \wedge \neg Q)$
- $(\forall x)\{[\neg[\neg P(x) \vee \neg Q(x)] \vee (\exists y)[S(x, y) \wedge Q(x)]]\} \wedge (\forall x)[P(x) \vee B(x)]$

2. 否定内移 把 $\neg$ 移到紧靠谓词的位置

- $\neg\neg P = P$
- $\neg(P \vee Q) = P \wedge \neg Q$
- $\neg(P \wedge Q) = \neg P \vee \neg Q$
- $\neg(\forall x)P = (\exists x)(\neg P)$
- $\neg(\exists x)P = (\forall x)(\neg P)$
- $(\forall x)\{[P(x) \wedge Q(x)] \vee (\exists y)[S(x, y) \wedge Q(x)]\} \wedge (\forall x)[P(x) \vee B(x)]$

3. 变量标准化

- 每个量词采用不同的变量
- $(\forall x)\{[P(x) \wedge Q(x)] \vee (\exists y)[S(x, y) \wedge Q(x)]\} \wedge (\forall w)[P(w) \vee B(w)]$

4. 消去存在量词 $\exists$

- 如果存在量词不在任何一个全称量词的辖域内, 则该存在量词不依赖于任何其它的变量, 因此可用一个新的个体常量代替 如将 $(\exists x)P(x)$ 化为 $P(A)$
- 如果存在量词是在全称量词的辖域内(如在公式 $(\forall y)((\exists x)P(x, y))$ 中, 变量x的取值依赖于变量y的取值)由Skolem函数 $f(y)$ 表示依赖关系
- $(\forall x)\{[P(x) \wedge Q(x)] \vee [S(x, f(x)) \wedge Q(x)]\} \wedge (\forall w)[P(w) \vee B(w)]$

5. 将公式化为前束形

- 把所有全称量词移到公式左边
- 使每个量词的辖域包含这个量词后面的整个部分
- $(\forall x)(\forall w)\{[P(x) \wedge Q(x)] \vee [S(x, f(x)) \wedge Q(x)]\} \wedge [P(w) \vee B(w)]$

6. 化为合取范式

- $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$
- $(P \wedge Q) \vee (P \wedge R) = P \wedge (Q \vee R)$
- $(\forall x)(\forall w)\{[P(x) \vee S(x, f(x))] \wedge Q(x) \wedge [P(w) \vee B(w)]\}$

7. 略去全称量词

- $[P(x) \vee S(x, f(x))] \wedge Q(x) \wedge [P(w) \vee B(w)]$

8. 消去合取符号 $\wedge$, 用子句集表示

- $P \wedge Q$ 表示为 P, Q
- 子句集为: $P(x) \vee S(x, f(x)), Q(x), P(w) \vee B(w)$

9. 子句变量标准化

- 使每个子句中的变量符号不同
- 子句集为: $P(x) \vee S(x, f(x)), Q(y), P(w) \vee B(w)$

归结原理

定义 匹配: 两个表达式匹配当且仅当**其语法是等价的**

合一就是寻找对变量的**置换**而使表达式一致的过程

- 为了使公式 $P(x, f(y), B)$ 与 $P(x, f(B), B)$ 匹配, 可以用常量B代替变量y
- 称为通过置换 $\{B/y\}$ 就可使上述公式集合一
- 置换可用有序对的集合 $s = \{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$ 表示
 - t_i/v_i 表示将表达式中所有的变量 v_i 都用项 t_i 代替
 - t_i 可以是变量、常量或函数

定义 若 P 是原子谓词公式, 则称 P 与 $\neg P$ 为**互补文字**

为将归结原理应用于含有变量的子句，应找出一个**置换**，作用于给定的两个子句，使它们包括**互补**的文字,然后才能进行归结

- 对于子句集 $S = \{P(x) \vee Q(x), \neg P(A) \vee R(y)\}$
- 通过置换 $s = \{A/x\}$ 变为 $\{P(A) \vee Q(A), \neg P(A) \vee R(y)\}$
- 此时进行归结，结果为 $Q(A) \vee R(y)$

定义 归结式

- C_1 和 C_2 是两个没有相同变量的子句，分别表示成文字集合 $\{L_i\}$ 和 $\{M_i\}$
- $\{l_i\}$ 是 $\{L_i\}$ 的一个子集， $\{m_i\}$ 是 $\{M_i\}$ 的一个子集
- 若**置换s**是 $\{l_i\}$ 和 $\{\neg m_i\}$ 的最简合一者
- 则称 $C_{12} = \{\{L_i\} - \{l_i\}\}_s \vee \{\{M_i\} - \{m_i\}\}_s$ 为 C_1 和 C_2 的归结式
- **注：不能同时消去两个互补文字**

归结原理的应用

应用归结原理进行定理证明

- 设要被证明的定理如下： $A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow B$
- 1. 首先否定结论 B ，并将否定后的公式 $\neg B$ 与前提公式集组成如下形式的谓词公式： $G = A_1 \wedge A_2 \wedge \dots \wedge A_n \wedge \neg B$
- 2. 求谓词公式 G 的子句集 S (也可分别求 A_i 和 $\neg B$ 的子句集再取并集)
- 3. 对 S 中的子句进行归结，并将得到的归结式并入 S 中
- 4. 若出现**NIL**，则证明定理成立；若无法得到**NIL**即停止，则证明定理不成立

例 已知：某些病人喜欢所有的医生；没有一个病人喜欢任意一个骗子；求证：任意一个医生都不是骗子

1. 知识表示
 - $P(x)$: x 是病人
 - $D(x)$: x 是医生
 - $Q(x)$: x 是骗子
 - $L(x, y)$: x 喜欢 y
 - $A_1: \exists x(P(x) \wedge \forall y(D(y) \rightarrow L(x, y)))$
 - $A_2: \forall x(P(x) \rightarrow (\forall y(Q(y) \rightarrow \neg L(x, y))))$
 - $B: \forall x(D(x) \rightarrow \neg Q(x))$
2. 分别求 A_1 、 A_2 、 $\neg B$ 的子句集，并取并集 S
 - $A_1: P(a), \neg D(y) \vee L(a, y)$
 - $A_2: \neg P(x) \vee \neg Q(y) \vee \neg L(x, y)$
 - $\neg B: D(b), Q(b)$
 - $S = \{P(a), \neg D(y) \vee L(a, y), \neg P(x) \vee \neg Q(y) \vee \neg L(x, y), D(b), Q(b)\}$
3. 归结演绎序列
 - i. $P(a)$
 - ii. $\neg D(y) \vee L(a, y)$
 - iii. $\neg P(x) \vee \neg Q(y) \vee \neg L(x, y)$
 - iv. $D(b)$
 - v. $Q(b)$
 - vi. $L(a, b)$ 【由2、4, $s = \{b/y\}$ 】
 - vii. $\neg Q(y) \vee \neg L(a, y)$ 【由1、3, $s = \{a/x\}$ 】
 - viii. $\neg L(a, b)$ 【由5、7, $s = \{b/y\}$ 】
 - ix. **NIL** 【由6、8】

利用归结原理求取问题答案

1. 把已知前提条件用谓词公式表示出来，并化成相应的子句集，设该子句集的名字为 S_1

2. 把待求解的问题也用谓词公式表示出来，然后将其否定，并与一谓词 $ANSWER$ 构成析取式。谓词 $ANSWER$ 是一个专为求解问题而设置的谓词，其变量必须与问题公式的变量完全一致
3. 把 2. 中的析取式化为子句集，并把该子句集与 S_1 合并构成子句集 S
4. 对子句集 S 应用归结原理进行归结，在归结的过程中，通过合一，改变 $ANSWER$ 中的变元
5. 如果得到归结式 $ANSWER$ ，则问题的答案即在 $ANSWER$ 谓词中

例 已知：任何兄弟都有同一个父亲； $John$ 和 $Peter$ 是兄弟； $John$ 的父亲是 $David$ ；求： $Peter$ 的父亲是谁？

1. 把前提用谓词公式表示并化为子句集 S_1
 - $Father(x, y)$: x 是 y 的父亲
 - $Brother(x, y)$: x 是 y 的兄弟
 - $F_1: \forall x \forall y \forall z (Brother(x, y) \wedge Father(z, x) \rightarrow Father(z, y))$
 - $F_2: Brother(John, Peter)$
 - $F_3: Father(David, John)$
 - $S_1 = \{\neg Brother(x, y) \vee \neg Father(z, x) \vee Father(z, y), Brother(John, Peter), Father(David, John)\}$
2. 把问题用谓词公式表示并与谓词 $ANSWER$ 作析取
 - 设 $Peter$ 的父亲是 u ，则有： $Father(u, Peter)$
 - $G: \neg Father(u, Peter) \vee ANSWER(u)$
3. 得到总子句集 S
 - $S = \{\neg Brother(x, y) \vee \neg Father(z, x) \vee Father(z, y), Brother(John, Peter), Father(David, John), \neg Father(u, Peter) \vee ANSWER(u)\}$
4. 归结演绎序列
 - i. $\neg Brother(x, y) \vee \neg Father(z, x) \vee Father(z, y)$
 - ii. $Brother(John, Peter)$
 - iii. $Father(David, John)$
 - iv. $\neg Father(u, Peter) \vee ANSWER(u)$
 - v. $\neg Brother(John, y) \vee Father(David, y)$ 【由1、3, $s = \{David/z, John/x\}$ 】
 - vi. $Father(David, Peter)$ 【由2、5, $s = \{Peter/y\}$ 】
 - vii. $ANSWER(David)$ 【由4、6, $s = \{David/u\}$ 】
5. 答案为 $David$

使用python进行逻辑推理

案例1：破案推理

已知：

1. 盗窃发生在晚上。
2. 犯罪嫌疑人要么是张三 ($ZhangSan$)，要么是李四 ($LiSi$)，但不可能是两人一起作案。
3. 目击者证实：罪犯在案发时戴着黑色手套。
4. 调查发现：张三家中有黑色手套，但李四没有。
5. 另有证据表明：罪犯作案时必须会撬锁，但张三不会撬锁，而李四是锁匠。
6. 由于犯罪现场没有强行进入的痕迹，说明罪犯是用撬锁进入的。

求：罪犯是谁？

1. 知识表示
 - 常量: $ZhangSan$ 、 $LiSi$
 - $Thief(x)$: x 是罪犯
 - $HasGloves(x)$: x 有黑手套
 - $CanPickLock(x)$: x 会撬锁

- $WearingGloves(x)$: x戴了黑手套
- $PickEntered$: 罪犯用撬锁进入
- $HappenAtNight$: 盗窃发生在晚上
- $WitnessSawGloves$: 目击者看见戴手套
- $IsLocksmith(x)$: x是锁匠

2. 形式化前提条件

- $HappenAtNight$
- $Thief(ZhangSan) \vee Thief(LiSi)$
- $\neg(Thief(ZhangSan) \wedge Thief(LiSi))$
- $\forall x(Thief(x) \rightarrow WearingGloves(x))$
- $HasGloves(ZhangSan)$
- $\neg HasGloves(LiSi)$
- $\neg CanPickLock(ZhangSan)$
- $IsLocksmith(LiSi) \rightarrow CanPickLock(LiSi)$
- $IsLocksmith(LiSi)$
- $PickEntered$
- $PickEntered \rightarrow \exists x(Thief(x) \wedge CanPickLock(x))$

3. 利用归结原理求取问题答案

- $\neg Thief(x) \vee ANSWER(x)$

4. 用python实现归结推理

```

# 简单归结推理模拟程序

# 定义基本事实
facts = \{
    "HasGloves": \{"ZhangSan"\},
    "CanPickLock": \{"LiSi"\},
    "IsLocksmith": \{"LiSi"\},
    "HappenAtNight": True,
    "PickEntered": True
\}

# 定义推理规则
def WearingGloves(x):
    """若是罪犯，则必须戴手套"""
    return x in facts["HasGloves"]

def CanPickLock(x):
    if x in facts["CanPickLock"]:
        return True
    if x in facts["IsLocksmith"]:
        return True
    return False

def IsThief(x):
    if not WearingGloves(x):
        return False
    if not CanPickLock(x):
        return False
    return True

# 候选人
candidates = ["ZhangSan", "LiSi"]

# 排除“两人都是罪犯”的情况
result = []
for x in candidates:
    if IsThief(x):
        result.append(x)

# 根据“不能两人都是罪犯”的前提，判断唯一结果
if len(result) == 1:
    print(f"罪犯是: \{result[0]\}")
elif len(result) == 0:
    print("无法判断罪犯")
else:
    print("前提矛盾，两个都有作案条件? ")

```

概率推理与贝叶斯网络

概率推理

定义 随机变量：表示可能世界中的不确定性。随机变量以大写字母开头

定义 概率分布：考虑随机变量 $Weather$ ，其定义域为 $\langle sunny, rainy, cloudy, snow \rangle$ ，每个可能取值的概率

- $P(Weather = sunny) = 0.6$
- $P(Weather = rain) = 0.1$
- $P(Weather = cloudy) = 0.29$
- $P(Weather = snow) = 0.01$

定义 先验概率：无条件概率

定义 联合概率分布：多个变量取值的所有组合的概率

Weather	sunny	rainy	cloudy	snow
Cavity = true	0.144	0.020	0.016	0.020
Cavity = false	0.576	0.080	0.064	0.080

定义 后验概率：条件概率

- 由无条件概率定义

$$P(a \mid b) = \frac{P(a,b)}{P(b)}$$

定义 乘法法则

$$P(a,b) = P(a \mid b)P(b) = P(b \mid a)P(a)$$

定义 链式法则

$$P(X_1,...,X_n) = \Pi_{i=1...n}P(X_i|X_1,...,X_{i-1})$$

定义 归一化方法：计算查询变量的概率分布，可以固定证据变量，然后在隐变量上求和并归一化

- 查询变量为X
- 证据变量为E，观察值为e
- 其余的未观测变量为Y

$$P(X|e) = \alpha P(X,e) = \alpha \Sigma_y P(X,e,y)$$

归一化常数 $\alpha = \frac{1}{P(e)}$

贝叶斯网络推理

定义 贝叶斯规则

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)}$$

定义 贝叶斯网络：一种使用简单局部分布（条件概率）描述复杂联合分布的技术

- 是一个有向无环图，也称之为图模型
- 用于表示变量之间的依赖关系
- 本质上，表示任何完全联合概率分布

语法

- **结点**：对应一个随机变量 X_i ，可以是已知的或者是待观测的
- **有向边**：表示连接的变量间的“直接影响”
- **条件概率分布**：量化父结点对该结点的影响

$$P(X_i | Parents(X_i))$$

- **条件概率表**：为每个变量指定其相对于父结点的条件概率
- **完全联合概率分布**：由局部条件概率分布的乘积进行定义

$$P(x_1, \dots, x_n) = \prod_{i=1 \dots n} P(x_i | parents(X_i))$$

示例 AlarmNetwork

- 描述：我在上班，邻居约翰打电话说我的报警器响了，但邻居玛丽不打电话。有时，它会受到轻微地震的影响。有窃贼吗？
- 布尔变量：Burglary, Earthquake, Alarm, JohnCalls, MaryCalls
- 网络拓扑
 - $Burglary \rightarrow Alarm$
 - $Earthquake \rightarrow Alarm$
 - $Alarm \rightarrow JohnCalls$
 - $Alarm \rightarrow MaryCalls$



$$P(j, m, a, \neg b, \neg e)$$

$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

$$= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998$$

$$= 0.00063$$

方法 构造贝叶斯网络

1. 节点：选择一组排好序的随机变量 $X_1, \dots, X_n$
2. 边：i从1到n，执行：
 - i. 从 $X_1, \dots, X_{i-1}$ 中选择最少的父节点，使得

$$P(X_i | Parents(X_i)) = P(X_i | X_1, \dots, X_{i-1})$$

- ii. 在每一个父结点与 X_i 之间插入一条边
- iii. 写出条件概率表 $P(X_i | Parents(X_i))$

贝叶斯枚举推理

接上一部分的示例

已知John和Mary都打来了电话，问出现盗贼的概率是多少？

根据归一化方法

$$P(B|j, m)$$

$$= \alpha P(B, j, m)$$

$$= \alpha \sum_e \sum_a P(B, e, a, j, m)$$

根据贝叶斯网络

$$= \alpha \sum_e \sum_a P(B) P(e) P(a|B, e) P(j|a) P(m|a)$$

$$= \alpha P(B) \sum_e P(e) \sum_a P(a|B, e) P(j|a) P(m|a)$$

$$= \alpha \times 0.001 \times \{[0.002 \times (0.95 \times 0.9 \times 0.7 + 0.05 \times 0.05 \times 0.01)] + [0.998 \times (0.94 \times 0.9 \times 0.7 + 0.06 \times 0.05 \times 0.01)]\}$$

$$= \alpha \times 0.00059224 \approx 0.284$$

搜索树构建与启发式搜索

搜索的形式化描述

定义 状态：对搜索算法和搜索环境当前所处情形的描述信息

定义 行动：算法从一个状态转移到另外一个状态所采取的行为

定义 状态转移模型：行动会产生的预期效果

定义 路径与代价：搜索过程中可以得到一个状态序列，这个状态序列被称为一条路径；每条路径对应一个代价

定义 目标测试：目标测试函数 $goal_test(s)$ 用于判断状态 s 是否为目标状态

搜索树的构建

- 根节点为初始状态
- 通过行动和状态转移模型，生成若干中间节点
- 状态之间的边 s 用行动和代价表示
- 叶节点表示目标状态

启发式搜索

搜索需要解决的基本问题

- 完备性
- 最优性
- 时间复杂度
- 空间复杂度
- b 代表搜索树一层最多节点数， m 代表搜索树层数

搜索方向

- 数据驱动：正向搜索
- 目的驱动：逆向搜索
- 双向搜索

无信息搜索

- 生成后继节点，判断是否目标状态
- DFS 、 BFS 、 UCS

启发式搜索

- $g(n)$ ：当前路径的代价
- $h(n)$ ：启发式函数，到目标节点的代价的估计
- $f(n) = g(n) + h(n)$
 - $g(n)$ 比重越大，越倾向 BFS
 - $h(n)$ 比重越大，启发性能越强

贪心搜索

- $f(n) = h(n)$
- 总是选取离目标节点最近的节点进行扩展
- 不具有完备性与最优性
- 时间复杂度和空间复杂度都是 $O(b^m)$

A^* 搜索

- $f(n) = g(n) + h(n)$
- 若对每个节点 n ，都有 $h(n) \leq h^*(n)$ ($h^*(n)$ 是节点 n 到目标节点的真实代价)，则称该 $h(n)$ 是可接受的
- 若 $h(n)$ 是可接受的，则该 A^* 搜索具有完备性和最优性

博弈论与对抗式搜索

博弈论

定义 参与者：决策主体

定义 策略：参与者可采取的行动方案

- 所有策略组合组成策略集
- 混合策略：通过概率分布选择不同策略
- 纯策略：每次行动都选择确定策略

定义 局势：所有参与者各自采取行动后形成的状态

定义 收益：各个参与者在不同局势下得到的利益

定义 规则：对参与者行动的先后顺序、参与者获得信息多少等内容的规定

分类

- 合作博弈：可以联盟
- 非合作博弈：彼此独立，不联盟
- 静态博弈：同时决策
- 动态博弈：先后决策，后决策者知道先决策者的选择
- 完全信息博弈：所有参与者均了解其他参与者的策略集、收益等信息
- 不完全信息博弈：并非所有参与者均掌握了所有信息

博弈的形式化描述

- S_0 ：初始状态，规范游戏开始时的情况。
- $\text{PLAYER}(s)$ ：定义此时该谁行动。
- $\text{ACTIONS}(s)$ ：返回此状态下的合法移动集合。
- $\text{RESULT}(s,a)$ ：转移模型，定义行动的结果。
- $\text{TERMINAL-TEST}(s)$ ：终止测试，游戏结束返回真，否则返回假。游戏结束的状态称为终止状态。
- $\text{UTILITY}(s,p)$ ：效用函数（也可称为目标函数或收益函数），定义游戏者 p 在终止状态 s 下的数值。在国际象棋中，结果是赢、输或平，分别赋予数值+1，0，或 1/2。有些游戏可能有更多的结果，例如双陆棋的结果是从 0 到+192。零和博弈是指在同样的棋局实例中所有棋手的总收益都一样的情况。国际象棋是零和博弈，棋局的收益是 0+1，1+0 或 1/2 + 1/2。“常量和”可能是更好的术语，但称为零和更传统，可以将这看成是下棋前每个棋手都被收了 1/2 的入场费。

Minimax搜索

两个决策者之间进行零和博弈，针对场上“分数”，一方想要最大化分数，一方想要最小化分数

算法步骤

1. $\text{value}(\text{state})$

- 功能：评估当前状态的得分。
- 逻辑：
 - 如果状态是终止状态（游戏结束），返回该状态的效用值（utility）。
 - 如果当前玩家是 **MAX**（最大化方），调用 $\text{max-value}(\text{state})$ 并返回结果。
 - 如果当前玩家是 **MIN**（最小化方），调用 $\text{min-value}(\text{state})$ 并返回结果。

2. $\text{max-value}(\text{state})$

- 功能：计算 **MAX** 玩家的最优决策值。
- 逻辑：
 - 初始化 $v = -\infty$ （负无穷）。
 - 对每个后继状态（successor），递归调用 $\text{value}(\text{successor})$ ，并更新 v 为最大值。
 - 返回最终的最大值 v 。

3. min-value(state)

- **功能**：计算 **MIN** 玩家的最优决策值。
- **逻辑**：
 - 初始化 $v = +\infty$ （正无穷）。
 - 对每个后继状态（successor），递归调用 `value(successor)`，并更新 v 为最小值。
 - 返回最终的最小值 v 。

$\alpha\beta$ 剪枝

- 当节点N以下的搜索完成（或剪枝中止）时，更新N的父节点的 α 或 β 值
- 当一个MAX节点N的 α 值 $\geq$ N 的MIN祖先节点的 β 值，则节点 N 以下的搜索中止
- 当一个MIN节点N的 β 值 $\leq$ N 的MAX祖先节点的 α 值，则节点 N 以下的搜索中止

算法步骤

1. max-value(state, α , β)

- **功能**：计算 **MAX** 玩家的最优决策值（带 Alpha-Beta 剪枝）。
- **参数**：
 - `state`：当前游戏状态。
 - α ：当前已知的 MAX 最佳值（初始为 $-\infty$ ）。
 - β ：当前已知的 MIN 最佳值（初始为 $+\infty$ ）。
- **逻辑**：
 - 初始化 $v = -\infty$ 。
 - 对每个后继状态（successor）：
 - 递归调用 `value(successor,  $\alpha$ ,  $\beta$ )`，更新 $v = \max(v, \text{value})$ 。
 - **剪枝条件**：如果 $v \geq \beta$ ，直接返回 v （无需继续搜索）。
 - 更新 $\alpha = \max(\alpha, v)$ 。
 - 返回 v 。

2. min-value(state, α , β)

- **功能**：计算 **MIN** 玩家的最优决策值（带 Alpha-Beta 剪枝）。
- **参数**：同 `max-value`。
- **逻辑**：
 - 初始化 $v = +\infty$ 。
 - 对每个后继状态（successor）：
 - 递归调用 `value(successor,  $\alpha$ ,  $\beta$ )`，更新 $v = \min(v, \text{value})$ 。
 - **剪枝条件**：如果 $v \leq \alpha$ ，直接返回 v （无需继续搜索）。
 - 更新 $\beta = \min(\beta, v)$ 。
 - 返回 v 。

- **剪枝优化**：通过 α 和 β 的传递与比较，跳过对无效分支的搜索，提升算法效率。
- **递归终止**：当 v 突破对方当前的最优值边界（ $v \geq \beta$ 或 $v \leq \alpha$ ）时，立即终止当前分支的进一步计算。

资源限制

- 在真实游戏中，搜索空间十分庞大
- 限制搜索树的深度，用截断测试取代终止测试，用启发式评估函数取代效用函数